

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Implement Modules in NodeJS

Student Name

Ch. Venu Gopal Reddy

Roll No.

A24126552078

Date

College

Anil Neerukonda Institute of Technology and
Sciences(ANITS)

Course Outcome

CO1

Course Outcome: CO2

1. TITLE

Implement Modules in NodeJS

2. AIM

To implement and demonstrate the use of modules in NodeJS by creating separate JavaScript files and executing them using NodeJS.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basic structure of a NodeJS program.
- To understand how JavaScript files can be organized into separate modules.
- To execute JavaScript files using NodeJS.
- To observe the use of NodeJS global objects and built-in features.
- To understand how separate files can be used as part of a NodeJS project.

4. THEORY

NodeJS provides a runtime environment for executing JavaScript outside a web browser. A NodeJS application can be organized into multiple JavaScript files or modules, where each file can contain related code.

Modules help organize programs into smaller and manageable files. NodeJS also provides several global objects and features that are available during program execution, such as console, process, Buffer, __dirname, __filename, setTimeout() and setInterval().

- console is used to display messages and output.
- process provides information and control over the current NodeJS process.
- Buffer is used to work with binary data.
- __dirname gives the path of the current directory.
- __filename gives the path of the current file.
- setTimeout() executes a function after a specified delay.
- setInterval() repeatedly executes a function at a specified interval.

5. ALGORITHM / PROCEDURE

1. Create a NodeJS project folder.
2. Create the required JavaScript module files inside the folder.
3. Write the required NodeJS code in the module files.
4. Save all JavaScript files.
5. Open the project folder in Visual Studio Code.
6. Open the VS Code terminal.
7. Run the required JavaScript file using the NodeJS command.

8. Observe the output displayed in the terminal.
9. Verify the file structure and execution output.

6. PROGRAM

The NodeJS program is implemented using the JavaScript files provided for this experiment. The files are kept as separate modules in the project folder.

6.1 config.js

```
// config.js
global.APP_NAME = "ShopEasy";
global.APP_VERSION = "1.0.0";
```

6.2 main.js

```
//1.global example

global.loginCount = 0;

function userLoggedIn() {
    global.loginCount++;
    console.log(`Total logins: ${global.loginCount}`);
}

userLoggedIn();
userLoggedIn();
userLoggedIn();

// ex:2 share data
require("./config.js");
console.log(`Welcome to ${APP_NAME} v${APP_VERSION}`);

// 2.console Example
const user = {
    Name: "Venu",
    age: ""
};

console.log("User object:", user);
console.table(user);
console.error("User age is missing!");

// ex:2
console.time("loopTime");

for (let i = 0; i < 100; i++) {}

console.timeEnd("loopTime");
```

```

//3.process example:
console.log(process.pid);
console.log("Current folder:", process.cwd());

//4.buffer example:
console.log("====Buffer example below====");
const buf1 = Buffer.from("Node");
const buf2 = Buffer.from(" JS");
const combined = Buffer.concat([buf1, buf2]);
console.log("combined buffer of buf1,buf2:",combined.toString());
// buffer example2
const buf = Buffer.from("Hello");
console.log(buf);
//buffer example3
const buf3 = Buffer.alloc(5);
console.log("Buffer3:" ,buf3);
const buf4 = Buffer.alloc(5, "A");
console.log("Buffer4:",buf4.toString());

//5.__dirname example:
const path = require("path");

const filePath = path.join(__dirname,"text");
console.log("Full file path:", filePath);

//6.__filename example:
console.log(`This file's folder: ${__filename}`);

//7.setTimeout
console.log("====Set Timeout examples below====");
console.log("Waiting...");

setTimeout(() => {
    console.log("2 seconds passed!");
}, 2000);

// setTimeoutExample2.js
console.log("Order placed!");

setTimeout(() => {
    console.log("Your order has been shipped 📦");
}, 3000);

console.log("You can continue browsing...");

//8.setInterval example
let count = 0;
console.log("====Set interval example below====");
const timer = setInterval(() => {

    console.log(`Tick ${count}`);

```

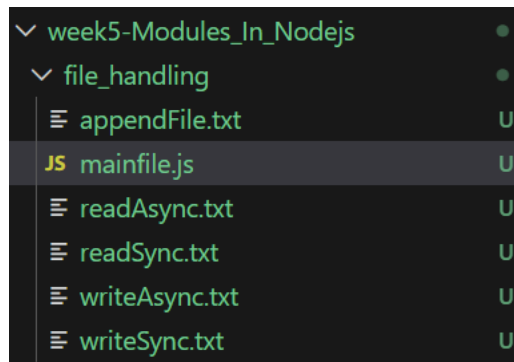
```
count++;

if (count === 3) {
    clearInterval(timer); // stop after 3 ticks
    console.log("Stopped!");
}
}, 5000);
```

7. HOW TO RUN THE PROGRAM

- Open the project folder in Visual Studio Code.
- Open the terminal in VS Code using Terminal → New Terminal.
- Make sure NodeJS is installed and available in the terminal.
- Navigate to the folder containing the JavaScript files.
- Run the required JavaScript file using: node filename.js
- Observe the program output in the terminal.

FILE STRUCTURE SCREENSHOT



8. OUTPUT

Paste the actual VS Code terminal output screenshots below.

Output : Program Execution: Modules/Nodejs Output

```
PS F:\full-stack-web-development\02-Record\week5-Modules_In_Nodejs\Global_object_demo> node main.js
Total logins: 1
Total logins: 2
Total logins: 3
Welcome to ShopEasy v1.0.0
User object: { Name: 'Venu', age: '' }



| (index) | Values |
|---------|--------|
| Name    | 'Venu' |
| age     | ''     |



User age is missing!
loopTime: 0.011ms
8212
Current folder: F:\full-stack-web-development\02-Record\week5-Modules_In_Nodejs\Global_object_demo
=====Buffer example below=====
combined buffer of buf1,buf2: Node JS
<Buffer 48 65 6c 6c 6f>
Buffer3: <Buffer 00 00 00 00 00>
Buffer4: AAAAAA
Full file path: F:\full-stack-web-development\02-Record\week5-Modules_In_Nodejs\Global_object_demo\text
This file's folder: F:\full-stack-web-development\02-Record\week5-Modules_In_Nodejs\Global_object_demo\main.js
=====Set Timeout examples below=====
Waiting...
Order placed!
You can continue browsing...
=====Set interval example below=====
2 seconds passed!
Your order has been shipped 📦
Tick 0
Tick 1
Tick 2
Stopped!
```

9. RESULT

Thus, the NodeJS program was implemented using separate JavaScript files/modules and executed successfully using NodeJS. The program structure and terminal output were verified.

10. CONCLUSION

The experiment provides an understanding of organizing JavaScript code in NodeJS modules and executing NodeJS programs from the terminal.